

MERGE SORT

- Merge Sort is a sorting algorithm that uses the Divide and Conquer strategy.
- It divides the problem into smaller sub problems, sorts them recursively, and finally merges the sorted sub arrays.
- Merge sort takes an input array with n elements and sorts the elements using the Divide and Conquer strategy.
- It consists of the following steps:
 1. **Divide:** The given array is divided into two equal halves. This process continues until each sub array contains only one element.
 2. **Conquer:** The divided sub arrays are sorted recursively by comparing their elements.
 3. **Combine:** The two sorted sub arrays are merged to form a single sorted array.

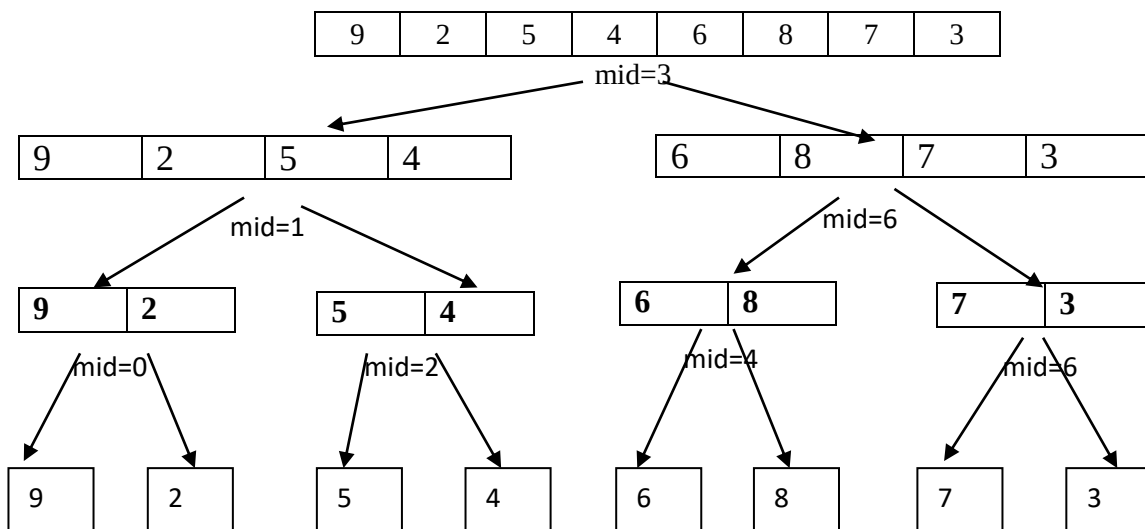
Example: Consider the input array:9,2,5,4,6,8,7,3

Sol: Let's perform Merge sort step by step

Given elements 9,2,5,4,6,8,7,3

Step 1: Apply Divide

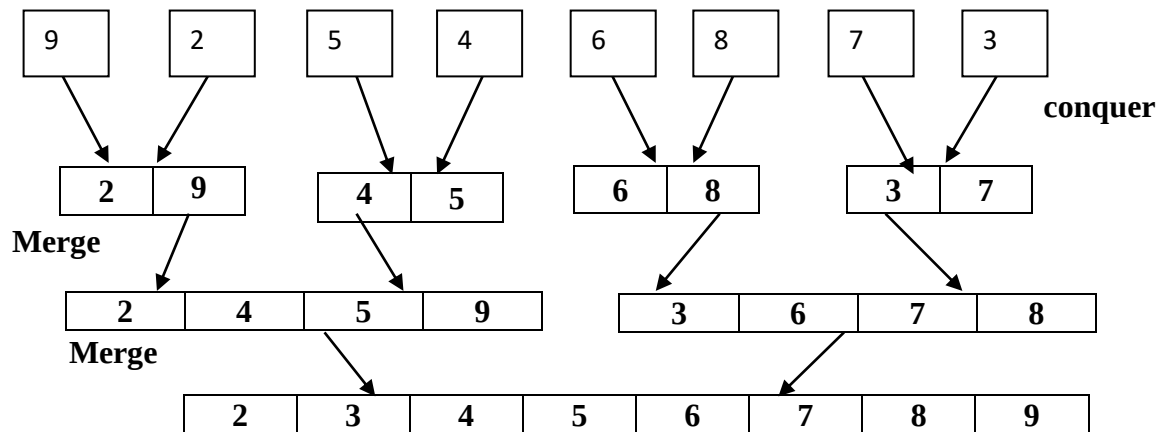
Divide array into two halves. $\text{Mid} = (\text{low} + \text{high}) / 2$



Step2: Apply Conquer and combine

Conquer: Compare elements and arrange in sorted order .

Combine: Merge the sorted sub arrays.



Therefore Merged Sorted array is:
[2,3,4,5,6,7,8,9]

Algorithm for Divide and Merge operation:

Algorithm divide(low, high)

```
{
  //low= Index points to first element of array
  //high=index points to last element of array
  low<-0
  high<-n-1
  if(low<high)then
  {
    mid=(low+high)/2 //split the list into two halves
    divide(low,mid)
    divide(mid+1,high)
    merge(A,low,mid,high)
  }
}
```

Algorithm merge(A,low,mid,high)

```
{
  // A is an array list
  // B is an array to store result(merged array)
  while(i<=mid and j<=high)do
  {
    if(A[i]<=A[j])then
    {
      B[k]=A[i]
      k++
      i++
    }
    else
    {
      B[k]=A[j]
      k++
    }
  }
}
```

```

        j++
    }
}
while(i<=mid)
{
    B[k]=A[i]
    i++
    K++
}
while(j<=mid)
{
    B[k]=A[j]
    j++
    K++
}
}

```

RECURRENCE RELATION FOR MERGE SORT:

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

Therefore standard recurrence equation for Merge sort is:

$$T(n) = 2T(n/2) + n$$

Here $T(n)$ = Time taken to sort n elements

$T(n/2)$ = Time to sort 2 sub array of size $n/2$

n = Merge the 2 sub array it takes $O(n)$

Calculate Time complexity:

General recurrence form is:

$$T(n) = aT(n/b) + f(n)$$

Standard recurrence equation is:

$$T(n) = 2T(n/2) + n$$

Here $a=2$, $b=2$, $f(n)=n$

Case 1: $a < b^d$ $d=1$

$$2 < 2^1$$

$$2 < 2 \text{ [F]}$$

Case 2: $a = b$

$$2 = 2 \text{ [T]}$$

Case 3: $a > b^d$

$$2 > 2^1$$

$$2 > 2 \text{ [F]}$$

Apply case 2 time complexity formula:

$$T(n) = \theta(n^d \log n)$$

$$= \theta(n^1 \log n)$$

$$T(n) = \theta(n \log n)$$

Time complexity for merge sort:

1. Best case: $O(n \log n)$
2. Average case: $O(n \log n)$
3. Worst case: $O(n \log n)$

QUICK SORT

- Quick Sort is a divide-and-conquer sorting algorithm.

Steps:

1. **Divide:** Choose an element as a **pivot** and partition the array around it:
 - Elements smaller than pivot → left side
 - Elements greater than pivot → right side
2. **Conquer:** Recursively apply Quick Sort to the left and right sub arrays.
3. **Combine:** No separate merging is required. Once the sub arrays are sorted, the entire array is sorted.